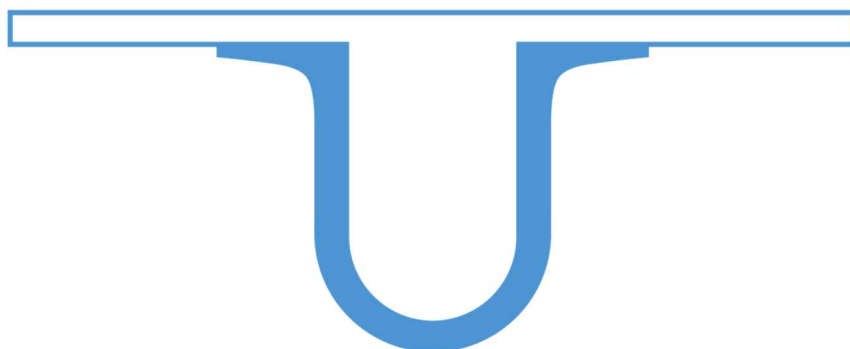




FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA



Final Delivery

***Implementation of a Public Key
Infrastructure***

Work done by:

João António Figueiredo de Macedo

2021220627

Lucas Miguel Simões Loberto

2021219107

Index

Introduction	3
Contextualization	4
Specification and Design	5
Initialization and Application Configuration	5
Reverse Proxy with NGINX.....	5
Dockerized Services	6
Home Page (Welcome Page)	6
Registration Process	7
Login Process.....	8
User Dashboard	9
Add Friends and Manage Requests	9
Secure Chat Interface.....	10
Implementation of the PKI	11
Software solutions	11
NGINX Configuration	11
Docker Configuration.....	13
Database Configuration	14
PKI Certificate Creation and Implementation	15
Importance of the Certificates.....	16
Role in PKI Implementation	16
Implementation of PKI for Registration and Login	17
Implementation of PKI for Messaging	19
Certificate and Key Generation	19
Establishing the Foundations of PKI for Messaging	21
Message Retrieval and Decryption	24
Cryptographic Guarantees and Security Features	25
Final thoughts	26
Characteristics of the PKI implemented	26
Limitations of the Current PKI Implementation	28
Potential Improvements for the Current PKI Implementation	30
Applicability and Use Cases of the Developed PKI	31
Conclusion	33

Introduction

The ever-increasing reliance on digital systems for communication, commerce, and data storage has elevated the importance of robust information security frameworks. In a world where sensitive data is constantly at risk from cyber threats, ensuring confidentiality, integrity, and authenticity is not merely an option but a necessity. Public Key Infrastructures (PKI) have emerged as a cornerstone of modern security practices, offering a reliable foundation for safeguarding information and verifying identities.

This project delves into the practical application of cryptographic technologies to establish a secure digital environment. By addressing prevalent security challenges such as data interception, identity fraud, and unauthorized access, the work emphasizes the critical role of PKI in fortifying communication channels. Through this initiative, we aim to showcase how cryptography can be leveraged to confront real-world security concerns effectively, contributing to the broader goal of building trust in digital interactions.

Furthermore, the significance of PKI extends beyond securing communications. It serves as a critical enabler for trust in various domains, including e-commerce, financial transactions, and digital identity management. By ensuring the authenticity of entities and the integrity of data, PKI provides a robust framework that underpins the digital economy, making it indispensable in today's interconnected world. This project aims to not only implement a secure infrastructure but also to explore its potential future in addressing broader security challenges.

Contextualization

This project revolves around the development of a secure web-based platform that functions as a chat application for its users. The implementation leverages a Public Key Infrastructure (PKI) to ensure robust security features across multiple layers of the system. The main components of this implementation include:

I. HTTPS with PKI

To ensure secure communication between the server and the users, HTTPS was implemented using SSL/TLS certificates. A custom Certificate Authority (CA) was used to issue and validate certificates, ensuring encrypted and authenticated data transmission. This approach protects against eavesdropping and man-in-the-middle attacks, providing users with a secure browsing experience.

II. Login with PKI

The login process integrates PKI to authenticate users securely. During registration, each user receives a unique certificate. At login, the system validates the user's credentials and checks the authenticity of their certificate. This dual validation ensures that only authorized users can access the platform, adding an extra layer of security to the login process.

III. Secure Message Exchange

To guarantee the confidentiality, integrity, and authenticity of messages exchanged between users, encryption and digital signatures were implemented. Messages are encrypted using the recipient's public key, ensuring that only the intended recipient can access them. Additionally, messages are also encrypted with the sender's public key, allowing the sender to securely view the messages they have sent. Each message is signed using the sender's private key, enabling the recipient to verify the sender's identity and ensuring that the message has not been tampered with during transmission. This dual encryption and signing process ensures a highly secure and trustworthy communication channel within the chat application.

By integrating these PKI-based mechanisms, the project demonstrates a practical application of cryptographic principles, emphasizing the importance of security in modern digital communications. The resulting system not only highlights the effectiveness of PKI in real-world scenarios but also provides a foundation for further enhancements and scalability.

Specification and Design

The implementation of this project required a detailed configuration of the system components to ensure a secure and functional environment. This section outlines the core design and specifications for the PKI-enabled platform.

Initialization and Application Configuration

The main application is powered by Flask, initialized with CSRF protection and custom configurations defined in the *nginx.conf* file. Logging mechanisms were implemented to capture application events and errors, ensuring traceability and debugging capabilities. The application is designed to run in a multi-threaded mode on port 5000, making it capable of handling multiple simultaneous requests efficiently.

Reverse Proxy with NGINX

NGINX is an open-source, high-performance web server and reverse proxy server that plays a critical role in the PKI implementation for this project. It acts as an intermediary layer to manage HTTP and HTTPS traffic, enabling secure and efficient communication between clients and the Flask application. By integrating NGINX with the PKI infrastructure, the system ensures that all client-server interactions are encrypted and authenticated. The main benefits and configurations include:

1. Improved Security

- **Reverse Proxy:** Acts as a reverse proxy, concealing the internal architecture of the application and minimizing direct exposure of backend services.
- **SSL/TLS Encryption:** Enables the use of SSL/TLS as part of the PKI implementation, ensuring secure communication between clients and the server.

2. Load Balancing

- **Scalability:** While not implemented in this project, NGINX supports load balancing, which distributes incoming traffic across multiple servers to enhance performance and resilience.

3. Request Routing

- **Request Management:** Routes incoming requests to the appropriate backend service, specifically the Flask application running on localhost. For testing purposes, the entire system was evaluated in a local environment to ensure proper functionality before deployment.

Dockerized Services

The platform leverages Docker to simplify the deployment and management of its services, ensuring consistency across different environments. Docker allows each component of the application to run in isolated containers, making the system more portable and scalable. This implementation relies on a Docker Compose configuration that defines the two main services: the database and the web server. These services are designed to work together seamlessly while maintaining modularity.

The database service uses PostgreSQL to handle all user data and message storage. This includes credentials, encrypted messages, and associated metadata.

The web service combines Flask and NGINX to provide both the application logic and secure HTTP/HTTPS communication. Flask handles the backend functionality, including user authentication, message processing, and certificate management that we are going to see ahead. NGINX acts as a reverse proxy, managing incoming requests and ensuring they are encrypted using SSL/TLS certificates provided by the custom PKI.

By containerizing these services, the platform achieves a high degree of modularity and maintainability. Docker ensures that the application can be deployed consistently in any environment, whether for development, testing, or production. This robust and modular configuration ensures that the platform operates securely and efficiently while being adaptable for future requirements.

Home Page (Welcome Page)

The home page is the entry point of the website, accessible through <https://localhost/>. This page greets users with a secure message highlighting that the website is protected with HTTPS using a custom PKI. It provides two primary navigation options:



1. [Go to Registration](#): Redirects the user to the registration page for creating a new account.
2. [Go to Login](#): Redirects the user to the login page for accessing the system.

This page serves as a simple and intuitive starting point for both new and returning users.

Registration Process

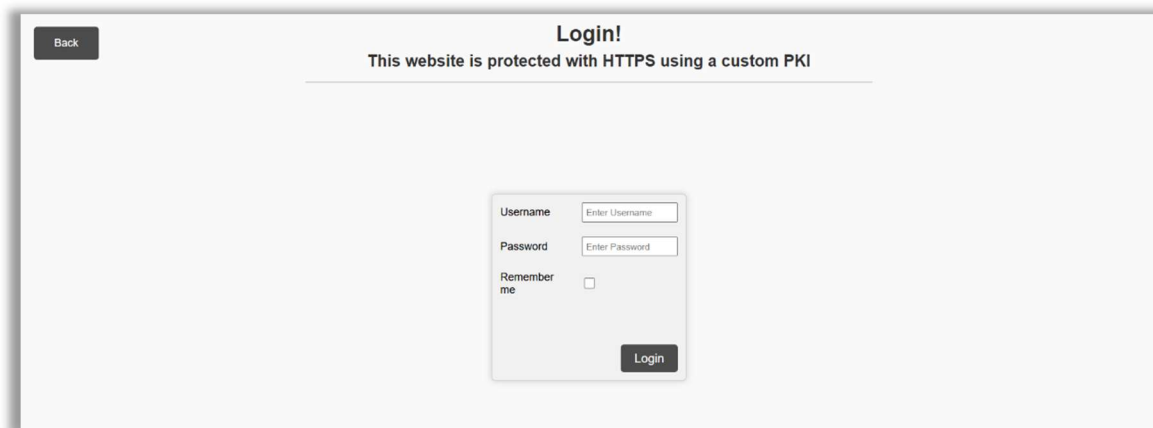
The screenshot shows a web interface for a login page. At the top left is a 'Back' button. The main heading is 'Login!'. Below it, a message states: 'This website is protected with HTTPS using a custom PKI'. The login form itself is centered and contains the following elements: a 'Username' label with an input field containing the placeholder text 'Enter Username'; a 'Password' label with an input field containing the placeholder text 'Enter Password'; a 'Remember me' checkbox which is currently unchecked; and a 'Login' button at the bottom right of the form.

The registration process is a critical component of the platform, integrating key elements of the Public Key Infrastructure (PKI). During registration, the system generates and manages cryptographic credentials for each user, establishing the foundation for secure interactions. Specifically, the platform creates two sets of RSA key pairs and corresponding certificates, one for user authentication and another for message encryption. These certificates are signed by the custom Certificate Authority (CA), ensuring their validity and trust within the system.

The private keys generated for message encryption are securely stored in a dedicated directory (Message_Userkeys). Each private key is encrypted using a password derived from the user's credentials, ensuring that only authorized individuals can access them. Similarly, the private keys generated for authentication purposes are securely stored in a separate directory (clients/). These keys are also encrypted with a user-derived password to maintain confidentiality. The public keys for message encryption are stored in the database, allowing the system to facilitate encrypted communication. However, the public keys for authentication are included in the corresponding user certificates and are not stored directly in the database, as they are used during the authentication process and validated against the certificate.

By leveraging PKI, the registration process securely handles the generation, storage, and distribution of cryptographic keys and certificates. This implementation ensures that every user has the necessary credentials to authenticate themselves and participate in encrypted communications, reinforcing the overall security and reliability of the platform.

Login Process



Back

Login!

This website is protected with HTTPS using a custom PKI

Username

Password

Remember me ☐

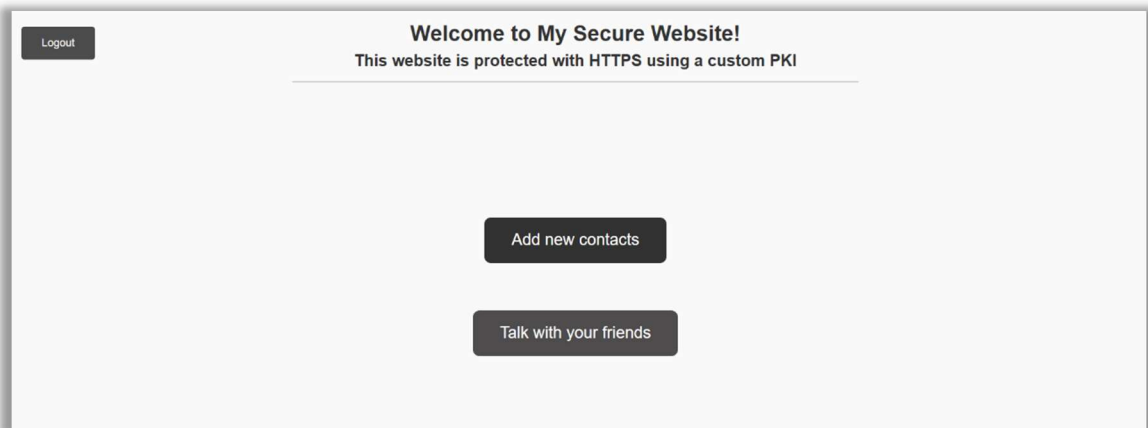
Login

The login process is a vital component of the platform, utilizing Public Key Infrastructure (PKI) to ensure secure and reliable user authentication. The system validates users through a dual-layer mechanism:

1. **Password Verification:** The user's password is hashed and compared to the securely stored hash in the database. This step confirms that the user-provided password matches the registered credentials.
2. **Certificate Validation:** A client certificate, issued during the registration process and signed by the platform's custom Certificate Authority (CA), is used to validate the user's identity. The server checks the certificate's signature and verifies its authenticity and integrity.

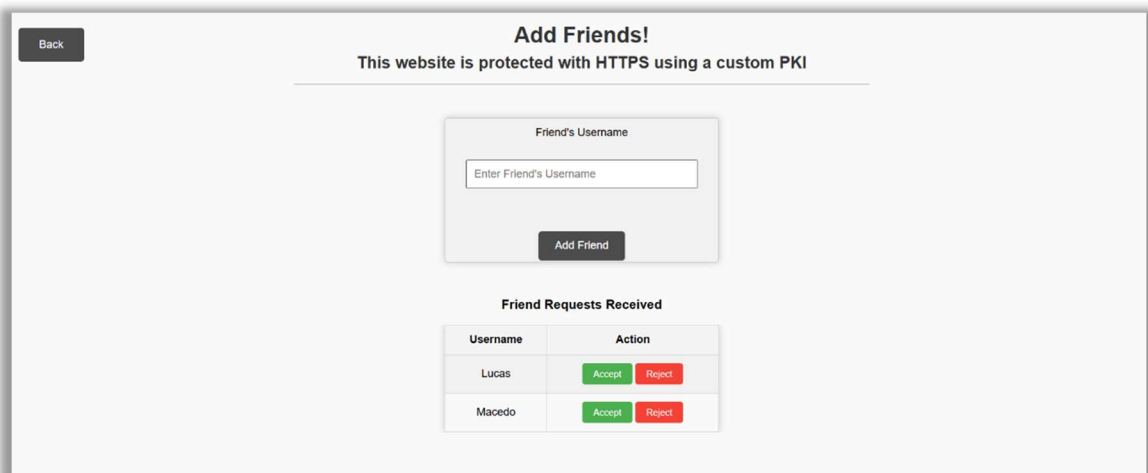
By leveraging PKI, the login process guarantees that authentication relies on both something the user knows (password) and something they possess (certificate). This integration enhances security by ensuring that only users with valid credentials and certificates can access the platform, preventing unauthorized access.

User Dashboard



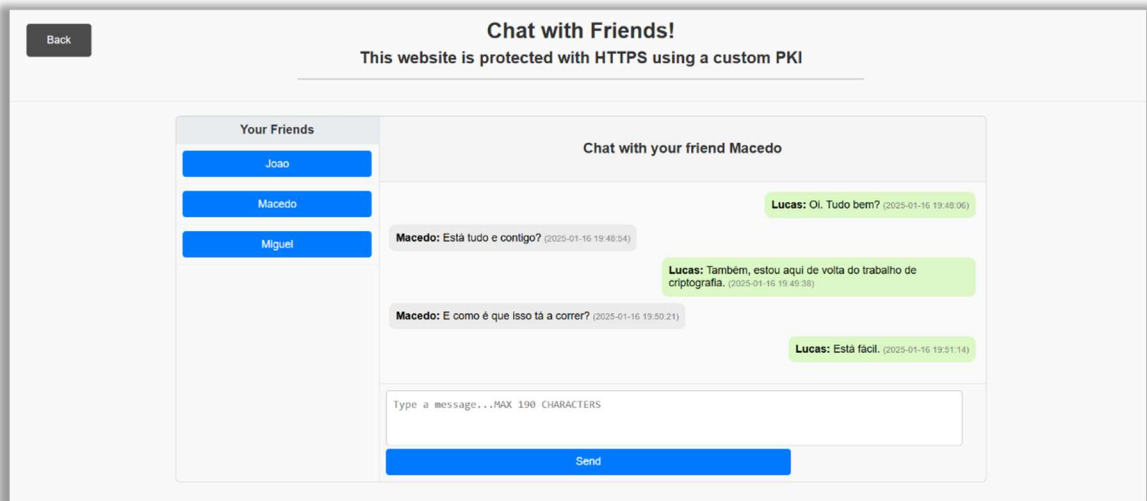
After a successful login, users are directed to the dashboard, which provides access to the core functionalities of the platform. From here, users can add new contacts or start secure conversations with their friends, utilizing the platform's PKI to ensure encrypted and authenticated communication. This dashboard acts as the central hub for managing interactions within the system.

Add Friends and Manage Requests



The "Add Friends" feature enables users to securely manage their social connections within the platform. Users can send friend requests by providing the username of the desired contact. The system performs several validations, such as ensuring the username exists, verifying that the user is not sending a request to themselves, and confirming that no friendship or pending request already exists. Incoming friend requests are displayed on the same page, allowing users to accept or reject them. Upon accepting a friend request, the system creates a secure connection between the users and establishes a new conversation if one does not already exist. Rejecting a request simply removes it from the list.

Secure Chat Interface



The "Secure Chat Interface" enables users to communicate with their friends through encrypted and authenticated messages. After selecting a friend, users can view the message history and exchange messages in real time. This feature heavily relies on the platform's PKI to ensure the confidentiality, integrity, and authenticity of the communication.

- 1. Friend Selection:** Users can select a friend from their contact list, which is dynamically retrieved from the database.
- 2. Message Encryption:** Before sending a message, the sender encrypts it using the recipient's public key, which is securely stored in the database, and signs it with their private key. Additionally, the sender encrypts a copy of the message with their own public key to allow viewing of their sent messages later. The sender's private key, that was previously securely stored in the server under the certificates/Message_Userkeys directory, is used for this signing process.
- 3. Message Decryption:** On the recipient's side, messages are decrypted using the recipient's private key, which is also stored securely in the certificates/Message_Userkeys directory. The platform verifies the sender's signature to ensure message authenticity and integrity.

This implementation ensures that all messages are securely transmitted and stored, preventing unauthorized access and tampering, thanks to the robust integration of PKI in the messaging process.

Implementation of the PKI

In this chapter, we detail the practical implementation of the Public Key Infrastructure (PKI). This includes the integration of software solutions, a step-by-step description of the development process, and the configuration of the system to ensure security and functionality. By combining cryptographic principles with modern development practices, this implementation bridges theoretical design with practical application, resulting in a robust and scalable platform.

Software solutions

The Public Key Infrastructure (PKI) is implemented through a combination of Python code, OpenSSL commands, and integration with Flask for seamless certificate management. Below is a breakdown of the key aspects of the PKI implementation in the code:

NGINX Configuration

The *nginx.conf* file sets up a secure and functional environment for the PKI-based system. Each component of the configuration contributes to the enforcement of HTTPS, the management of client certificates, and the secure routing of requests to the backend application.

```
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml application/xml+rss text/javascript;

    server {
        listen 443 ssl;
        server_name localhost;

        ssl_certificate /certificates/server.crt;
        ssl_certificate_key /certificates/server.key;

        ssl_client_certificate /certificates/ca.crt;
        ssl_verify_client optional;

        location /static/ {
            alias /app/static;
        }

        location / {
            proxy_pass http://127.0.0.1:5000;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header X-Client-Cert $ssl_client_cert;
        }
    }

    server {
        listen 80;
        server_name localhost;

        return 301 https://$host$request_uri;
    }
}
```

The following is a comprehensive overview of the configuration:

Worker Configuration

- **worker_processes auto;**
 - Dynamically sets the number of worker processes based on the number of CPU cores available. This ensures efficient resource usage and scalability.
- **worker_connections 1024;**
 - Defines the maximum number of simultaneous connections each worker can handle. This limits resource consumption while maintaining high availability.

Compression for Efficiency

- **gzip on;**
 - Enables GZIP compression for reducing the size of responses sent to clients, improving performance.
- **gzip_types:**
 - Specifies the types of files eligible for compression, such as:
 - text/plain: Plain text files.
 - text/css: Stylesheets.
 - application/json: JSON data.
 - application/javascript: JavaScript files.
 - Others related to web content.

HTTPS Server Block

- SSL Configuration:
 - **ssl_certificate and ssl_certificate_key**
 - Specify the paths to the server's public certificate and private key, enabling secure communication.
 - These files are generated using the PKI and are critical for establishing trust between the client and server.
 - **ssl_client_certificate**
 - Points to the CA's certificate, allowing the server to validate client certificates against the CA's trust chain.
 - **ssl_verify_client optional;**
 - Verifies client certificates if provided but does not require them. This enables flexibility, allowing both authenticated and unauthenticated clients to access specific resources.
- Static File Handling:
 - **location /static/ { alias /app/static/; }**
 - Maps requests for static assets (e.g., CSS, images, JavaScript) to the appropriate directory on the server, bypassing the backend for efficiency.
- Reverse Proxy:
 - **proxy_pass http://127.0.0.1:5000;**
 - Forwards all other requests to the Flask application running on localhost:5000.

- **proxy_set_header Host \$host;**
 - Passes the original Host header to the backend, preserving client request information.
- **proxy_set_header X-Real-IP \$remote_addr;**
 - Sends the client's IP address to the backend for logging or processing.
- **proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;**
 - Includes the entire chain of IP addresses for clients connecting through proxies.
- **proxy_set_header X-Forwarded-Proto \$scheme;**
 - Indicates whether the original request was HTTP or HTTPS.
- **proxy_set_header X-Client-Cert \$ssl_client_cert;**
 - Adds the client certificate as a header, enabling the backend to validate the certificate if necessary.

HTTP to HTTPS Redirection

- **server { listen 80; return 301 https://\$host\$request_uri; }**
 - Redirects all HTTP traffic on port 80 to HTTPS (port 443), ensuring all communication remains encrypted.

Security Features

- **TLS Protocols and Ciphers:**
 - By default, modern versions of NGINX enforce secure TLS protocols (e.g., TLSv1.2 and TLSv1.3).
 - Strong cipher suites are used to prevent vulnerabilities like weak encryption or outdated algorithms.
- **Certificate Validation:**
 - The client certificate is verified against the CA's certificate, ensuring only trusted certificates are accepted.

Docker Configuration

Docker plays a crucial role in managing the infrastructure for the PKI implementation, ensuring each component works seamlessly together. It was used to containerize the PKI infrastructure and its supporting components. This approach ensures consistency across environments and simplifies the deployment process.

Components in Docker:

- **Flask:** Hosts the backend logic.
- **NGINX:** Acts as a reverse proxy, managing HTTPS traffic and client certificate validation.
- **PostgreSQL:** Stores sensitive information such as user credentials, certificates, and logs.

- **Certificates Volume:** A shared volume stores the certificates used by both NGINX and Flask.

Docker Compose:

- Docker Compose is used to orchestrate all the services, including Flask, NGINX, and PostgreSQL.
- Each service is defined in the docker-compose.yml file, with appropriate configurations for networking and data sharing.

Benefits of Docker in the PKI Setup:

Docker provides several benefits for the PKI implementation. It ensures environment consistency by maintaining the same configuration across development, testing, and production. It supports scalability, allowing the backend to scale horizontally with minimal effort. Additionally, Docker enhances security by isolating services within containers, reducing the risk of system-wide vulnerabilities. Finally, it simplifies management, as all services can be started with a single command.

Database Configuration

The PostgreSQL database represents a critical component of the PKI-based system, meticulously designed to securely manage sensitive data.

The system comprises several tables, each serving a specific purpose. The Users table, for instance, stores information about each user, including a unique username, a salted and hashed password for security, and the user's public key in PEM format for encryption. The Friend Requests table is responsible for the management of friendship requests, recording the sender and receiver of each request and its status (e.g., pending or accepted). Once friendships are established, they are stored in the Friends table, which tracks user relationships along with timestamps of when they were created. The database also supports communication between users through the Conversations table, which ensures a unique conversation exists for each pair of users. Messages exchanged in these conversations are stored in the Conversation Messages table, which includes the encrypted content of each message, a version encrypted specifically for the sender, and a digital signature to ensure authenticity and integrity.

The database design incorporates several key features. The database enhances security by hashing and salting passwords and storing cryptographic data in secure formats. Furthermore, message encryption ensures that both sender and recipient can access their respective encrypted messages securely. The inclusion of timestamps in all tables enables the system to track the history of user interactions and activities.

This relational schema supports scalability, allowing for the seamless addition of more users, messages, and features without performance degradation. Its design also ensures consistency across all relationships and provides robust support for the cryptographic operations required in a PKI system.

PKI Certificate Creation and Implementation

The certificates stored in the certificates folder were manually created as part of the PKI infrastructure to enable secure communications, authentication, and trust within the platform. These certificates are critical components of the system's cryptographic foundation, ensuring the confidentiality, integrity, and authenticity of data exchanges. It is important to note that these certificates were generated for testing purposes only and do not represent certificates issued by a verified Certificate Authority (CA). As such, while they serve to demonstrate the functionality of a PKI, they are not suitable for production use. Below are the steps taken to implement the PKI and an explanation of their significance:

Generating the CA Private Key: The Certificate Authority (CA) private key was created using the command:

```
$ openssl genrsa -out certificates/ca.key 4096
```

This 4096-bit RSA key acts as the root of trust in the PKI hierarchy. It is used exclusively to sign other certificates, ensuring they can be trusted by the system.

Creating the CA Certificate: The CA certificate was generated with:

```
$ openssl req -x509 -new -nodes -key certificates/ca.key -sha256 -days 365 -out certificates/ca.crt
```

This certificate is self-signed and establishes the CA as the trusted authority. All other certificates in the system derive their trust from this CA certificate. The 365-day validity ensures the system remains operational within this timeframe, after which the certificate would need to be renewed.

Generating the Server Private Key: The server's private key was generated using the command:

```
$ openssl genrsa -out certificates/server.key 2048
```

This key is used by the server to establish secure connections with clients. By keeping this key private, the server can prove its identity during the TLS handshake, ensuring that communications are encrypted and secure.

Creating the Server Certificate Signing Request (CSR): The CSR for the server was created with:

```
$ openssl req -new -key certificates/server.key -out certificates/server.csr
```

The CSR was configured with information about the server, such as its common name (e.g., projetocripto), organization, and location. This step ensures that the server certificate accurately represents the server's identity. The Distinguished Name (DN) fields filled during this step provide essential details to uniquely identify the entity associated with the certificate.

Signing the Server Certificate with the CA: The CA signed the server certificate using:

```
$ openssl x509 -req -in certificates/server.csr -CA certificates/ca.crt -CAkey certificates/ca.key -CAcreateserial -out certificates/server.crt -days 365 -sha256
Certificate request self-signature ok
subject=C = PT, ST = Coimbra, L = Coimbra, O = UC, OU = DEI, CN = projetocripto
```

This signed certificate (server.crt) links the server's identity to the CA's root of trust. Clients that trust the CA certificate will inherently trust this server certificate, enabling secure HTTPS connections.

Importance of the Certificates

These certificates are essential for implementing the PKI and ensuring the secure operation of the platform:

- **CA Certificate:** Acts as the root of trust. Any certificate signed by this CA is considered trustworthy by the platform, enabling secure authentication and encryption.
- **Server Certificate:** Used to establish HTTPS connections, ensuring that data transmitted between clients and the server is encrypted. This prevents unauthorized parties from intercepting or altering communications.
- **Private Keys:** The private keys (both for the CA and the server) are securely stored in the certificates folder. They are never shared or exposed, which is critical to maintaining the integrity of the PKI. For example, the CA private key is used exclusively to sign certificates, while the server private key is used to prove the server's identity during communication.

Role in PKI Implementation

The process of generating and managing these certificates underpins the PKI implementation by establishing a trusted hierarchy with the CA at the top, enabling secure communication via HTTPS where data is encrypted end-to-end, providing authentication mechanisms where the server's identity is verified by its certificate, and supporting future extensions such as issuing client certificates for additional PKI features. This carefully planned certificate generation process ensures that the platform operates securely, protecting both user data and system integrity.

Implementation of PKI for Registration and Login

The implementation of PKI for registration and login introduces a secure and cryptographically robust process for user authentication and identity verification. This system leverages digital certificates issued by a custom Certificate Authority (CA) to ensure the authenticity of users and the integrity of their interactions.

During the registration process, a new user submits their details, and the system generates a unique private key for them. This key is securely stored on the client-side and is used to create a Certificate Signing Request (CSR) containing the user's information. The CSR is then sent to the server, where it is signed by the CA to issue a user-specific certificate. This certificate is stored securely on the server and optionally shared with the user for local use. It binds the user's identity to their cryptographic keys, enabling secure communication and authentication in the future. Additionally, the public key extracted from the certificate is stored in the database, allowing other users to encrypt messages directed to this user. The private key, secured through encryption, ensures that only the owner can decrypt messages or sign data, maintaining the confidentiality and authenticity of communications.

```
msg_private_key, msg_cert, public_key_pem = generate_certificate(username, usage="Messages")
save_private_key(username, msg_private_key, key)

conn = get_db()
cur = conn.cursor()
cur.execute(
    """
    INSERT INTO users (username, password, salt, msg_public_key)
    VALUES (%s, %s, %s, %s)
    """
    (username, key_hex, salt_hex, public_key_pem.decode('utf-8'))
)
conn.commit()
conn.close()

cert_dir = f"../certificates/clients/{username}"
if not os.path.exists(cert_dir):
    os.makedirs(cert_dir)

client_key = f"{cert_dir}/{username}-key.pem"
client_csr = f"{cert_dir}/{username}.csr"
client_cert = f"{cert_dir}/{username}-cert.pem"

try:
    subprocess.run(["openssl", "genrsa", "-out", client_key, "2048"], check=True)
    subprocess.run(["openssl", "req", "-new", "-key", client_key, "-out", client_csr, "-subj", f"/CN={username}"], check=True)
    subprocess.run(["openssl", "x509", "-req", "-in", client_csr, "-ca", "../certificates/ca.crt", "-CAkey", "../certificates/ca.key", "-CAcreateserial",
                    "-out", client_cert, "-days", 365], check=True)
except subprocess.CalledProcessError as e:
    logger.error(f"Error generating certificate for (username): {e}")
    return render_template("registration.html", message="Error generating certificate. Please try again.", message_type="error")

with open(client_cert, 'r') as cert_file:
    client_cert_data = cert_file.read()

encoded_cert = base64.b64encode(client_cert_data.encode('utf-8')).decode('utf-8')

return render_template("registration.html", message="User registered successfully.", message_type="success", client_cert=encoded_cert, username=username)
```

```
def generate_certificate(username, usage):
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
    )
    public_key = private_key.public_key()

    with open("/certificates/ca.key", "rb") as ca_key_file:
        ca_private_key = serialization.load_pem_private_key(
            ca_key_file.read(),
            password=None
        )
    with open("/certificates/ca.crt", "rb") as ca_cert_file:
        ca_cert = x509.load_pem_x509_certificate(ca_cert_file.read())
```

```

csr = x509.CertificateSigningRequestBuilder().subject_name(
    x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, f"({username}) ({usage})"),
    ])
).sign(private_key, hashes.SHA256())

cert = (
    x509.CertificateBuilder()
    .subject_name(csr.subject)
    .issuer_name(ca_cert.subject)
    .public_key(public_key)
    .serial_number(x509.random_serial_number())
    .not_valid_before(datetime.now(timezone.utc))
    .not_valid_after(datetime.now(timezone.utc) + timedelta(days=365))
    .add_extension(
        x509.KeyUsage(
            digital_signature=True,
            key_encipherment=(usage == "Messages"),
            content_commitment=True,
            data_encipherment=(usage == "Messages"),
            key_agreement=False,
            key_cert_sign=False,
            crl_sign=False,
            encipher_only=False,
            decipher_only=False
        ),
        critical=True,
    )
    .sign(ca_private_key, hashes.SHA256())
)

public_key_pem = cert.public_key().public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

return private_key, cert, public_key_pem

```

In order to initiate the login process, the user presents their certificate to the system, which is then validated against the CA's root certificate to ensure it was issued by a trusted authority. The server then verifies the certificate's validity, including its signature, expiration date, and the Common Name (CN) field, which should match the username. If the certificate passes all checks, the user is authenticated, and a secure session is established.

```

client_cert_base64 = request.form[f'({username}).cert']
client_cert = base64.b64decode(client_cert_base64).decode('utf-8')

try:
    with open("/tmp/client_cert.pem", "w") as cert_file:
        cert_file.write(client_cert)

    result = subprocess.run(
        ["openssl", "x509", "-in", "/tmp/client_cert.pem", "-noout", "-subject"],
        capture_output=True,
        text=True
    )
    subject = result.stdout.strip()

    if "CN =" in subject:
        username = subject.split("CN = ")[1].split(",")[0]
        session["user"] = username
    else:
        return render_template("login.html", message="Authentication failed. Please try again.", message_type="error")

```

This implementation provides several security benefits. It eliminates reliance on traditional passwords, thereby reducing vulnerabilities to attacks such as brute force and phishing. The use of certificates ensures strong user authentication and data integrity, as each certificate is unique, tamper-proof, and cryptographically bound to its owner. Additionally, this PKI-based system supports scalability, allowing seamless integration of advanced security features like encrypted messaging and role-based access control.

Implementation of PKI for Messaging

The implementation of the PKI for secure messaging involved the generation, storage, and management of certificates and private keys tailored for user communications. Below is an explanation of the approach and key steps executed.

Certificate and Key Generation

For each user registering on the platform, a dedicated RSA key pair and a corresponding certificate are generated specifically for message encryption and authentication.

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)
public_key = private_key.public_key()

with open("/certificates/ca.key", "rb") as ca_key_file:
    ca_private_key = serialization.load_pem_private_key(
        ca_key_file.read(),
        password=None
    )
with open("/certificates/ca.crt", "rb") as ca_cert_file:
    ca_cert = x509.load_pem_x509_certificate(ca_cert_file.read())

csr = x509.CertificateSigningRequestBuilder().subject_name(
    x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, f"{{username}} ({{usage}})"),
    ])
).sign(private_key, hashes.SHA256())

cert = (
    x509.CertificateBuilder()
    .subject_name(csr.subject)
    .issuer_name(ca_cert.subject)
    .public_key(public_key)
    .serial_number([x509.random_serial_number()])
    .not_valid_before(datetime.now(timezone.utc))
    .not_valid_after(datetime.now(timezone.utc) + timedelta(days=365))
    .add_extension(
        x509.KeyUsage(
            digital_signature=True,
            key_encipherment=(usage == "Messages"),
            content_commitment=True,
            data_encipherment=(usage == "Messages"),
            key_agreement=False,
            key_cert_sign=False,
            crl_sign=False,
            encipher_only=False,
            decipher_only=False
        ),
        critical=True,
    )
    .sign(ca_private_key, hashes.SHA256())
)
```

The `generate_certificate(username, usage="Messages")` function is a crucial part of the PKI implementation, responsible for creating a 2048-bit RSA private key for each user using the Python cryptography library. It then builds a Certificate Signing Request (CSR) containing user-specific details, such as the username and the intended usage, which in this case is set to "Messages." This CSR is signed using the Certificate Authority's (CA) private key, resulting in a certificate that ensures trust within the PKI infrastructure. The certificate is configured to be valid for 365 days and includes extensions like key usage to clearly define its role in message encryption and signing, further enhancing the platform's secure communication framework.

Storage of the Private Key

The private key, critical for decrypting messages and signing outbound communications, is stored securely on the server.

```
base_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "../certificates"))
user_keys_dir = os.path.join(base_dir, "Message_Userkeys")

user_dir = os.path.join(user_keys_dir, username)
os.makedirs(user_dir, exist_ok=True)

filename = os.path.join(user_dir, f"msg_{username}_private.pem")

with open(filename, "wb") as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.BestAvailableEncryption(password)
    ))
```

The `save_private_key(username, private_key, password)` function is designed to securely handle user private keys by encrypting them using a password derived from the user's registration password. The encrypted private key is then stored in a dedicated directory named `Message_Userkeys`, ensuring it remains isolated and protected on the server. The directory structure is organized to allow for easy identification and management of private keys for individual users, following the pattern `certificates/Message_Userkeys/{username}/`, which enhances the security and accessibility of these sensitive files.

Public Key Storage

The user's public key is stored in the database as part of the user's profile. This allows other users to retrieve the public key when they need to encrypt messages for the intended recipient.

```
msg_private_key, msg_cert, public_key_pem = generate_certificate(username, usage="Messages")
save_private_key(username, msg_private_key, key)

conn = get_db()
cur = conn.cursor()
cur.execute(
    """
    INSERT INTO users (username, password, salt, msg_public_key)
    VALUES (%s, %s, %s, %s)
    """,
    (username, key_hex, salt_hex, public_key_pem.decode('utf-8'))
)
conn.commit()
conn.close()
```

During the registration process, the public key is extracted from the user's certificate and stored in the database under the msg_public_key field. This approach ensures centralized access to public keys, allowing the system to retrieve them when needed while maintaining the overall integrity and security of the platform.

Establishing the Foundations of PKI for Messaging

The PKI integration during registration lays the groundwork for secure communication within the platform. This process ensures:

1. **Key Pair Generation:** A private-public key pair is created for each user, providing the basis for encryption and authentication.
2. **Secure Private Key Storage:** The private key is encrypted and stored securely in an organized directory structure on the server.
3. **Public Key Centralization:** The public key is stored in the database, allowing efficient access and management.

Message Sending and Storing

The process of sending a message is broken into key steps, which are combined here for simplicity:

Retrieving the Recipient and Sender Public Keys

The recipient's public key is fetched from the database (msg_public_key field), ensuring the message can be encrypted specifically for them.

```
cur.execute("SELECT msg_public_key FROM users WHERE username = %s", (friend,))
recipient_public_key_pem = cur.fetchone()[0]
recipient_public_key = serialization.load_pem_public_key(recipient_public_key_pem.encode('utf-8'))
```

Encrypting the Message for Both Parties

Recipient's Encryption: The recipient's public key encrypts the message, ensuring only the recipient can decrypt it.

Sender's Encryption: Simultaneously, the sender's public key encrypts the message, allowing the sender to access their sent messages securely later.

```
encrypted_message = recipient_public_key.encrypt(
    message.encode('utf-8'),
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

sender_encrypted_message = sender_public_key.encrypt(
    message.encode('utf-8'),
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

- **Algorithm:** RSA with a 2048-bit key size.
- **Mode:** OAEP (Optimal Asymmetric Encryption Padding) with SHA-256 as the hashing algorithm.

Message Signing

Retrieves the sender's private key from the server, decrypts it and signs the encrypted message, generating a digital signature. This signature provides proof of authenticity and ensures the message has not been altered.

```
private_key_path = os.path.join("/certificates/Message_Userkeys", username, f"msg_{username}_private.pem")
with open(private_key_path, "rb") as key_file:
    private_key = serialization.load_pem_private_key(
        key_file.read(),
        password=key
    )

signature = private_key.sign(
    encrypted_message,
    padding.PSS(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

- **Algorithm:** RSA-PSS (Probabilistic Signature Scheme).
- **Hashing:** SHA-256 ensures strong integrity checks.

Database Storage

The system employs a relational database to securely store messages and their associated metadata. Each message is recorded in the `conversation_messages` table, with the following fields:

- **Conversation ID:** Identifies the unique conversation between two users. This is fetched dynamically based on the users participating in the conversation.
- **Sender:** Indicates the username of the user who sent the message.
- **Message:** The actual message content, encrypted with the recipient's public key using RSA-OAEP and stored as binary data.
- **Signature:** The digital signature generated by the sender's private key, ensuring the message's authenticity and integrity.
- **Sender Encrypted Message:** The message content encrypted with the sender's own public key. This enables the sender to later decrypt and view the message they sent without relying on the recipient.

```
cur.execute("""
INSERT INTO conversation_messages (conversation_id, sender, message, signature, sender_encrypted_message)
VALUES (
    (SELECT conversation_id FROM conversations
     WHERE (user1 = %s AND user2 = %s) OR (user1 = %s AND user2 = %s)),
    %s, %s, %s, %s
)
""", (
    username, friend, friend, username,
    username,
    psycopg2.Binary(encrypted_message),
    psycopg2.Binary(signature),
    psycopg2.Binary(sender_encrypted_message)
))
```

Message Retrieval and Decryption

Retrieving messages involves a series of steps to ensure secure access and verification:

Fetching Messages from the Database

All messages exchanged in a conversation are fetched from the database, including metadata such as the sender, signature, and timestamps.

```
cur.execute("""
    SELECT sender, message, signature, sender_encrypted_message, sent_at FROM conversation_messages
    WHERE conversation_id = (
        SELECT conversation_id FROM conversations
        WHERE (user1 = %s AND user2 = %s) OR (user1 = %s AND user2 = %s)
    )
    ORDER BY sent_at
""", (username, friend, friend, username))

messages = [
    {
        "sender": msg[0],
        "encrypted_message": bytes(msg[1]),
        "signature": bytes(msg[2]),
        "sender_encrypted_message": bytes(msg[3]),
        "timestamp": msg[4]
    }
    for msg in cur.fetchall()
]
```

Decrypting the Message

For Received Messages: The recipient uses their private key to decrypt the message encrypted with their public key.

For Sent Messages: If the message was sent by the user, the system decrypts the sender-specific encryption using the sender's private key.

```
decrypted_messages = []
for msg in messages:
    try:
        if username == msg["sender"]:
            decrypted_message = private_key.decrypt(
                msg["encrypted_message"],
                padding.OAEP(
                    mgf=padding.MGF1(algorithm=hashes.SHA256()),
                    algorithm=hashes.SHA256(),
                    label=None
                )
            )
        else:
            decrypted_message = private_key.decrypt(
                msg["encrypted_message"],
                padding.OAEP(
                    mgf=padding.MGF1(algorithm=hashes.SHA256()),
                    algorithm=hashes.SHA256(),
                    label=None
                )
            )
    except:
```

Signature Verification

The sender's public key is retrieved to verify the digital signature. Any tampering with the message or signature invalidates the verification.

```
sender_public_key = get_sender_public_key(msg["sender"])

sender_public_key.verify(
    msg["signature"],
    msg["encrypted_message"],
    padding.PSS(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

Cryptographic Guarantees and Security Features

The implementation of this messaging system leverages the PKI infrastructure to provide essential cryptographic guarantees that ensure secure and reliable communication.

Confidentiality is achieved using RSA encryption with OAEP padding, which securely encrypts messages. This ensures that only the intended recipient, using their private key, can decrypt and access the message content, preventing unauthorized access by third parties.

Authentication is ensured by using RSA-PSS digital signatures. These signatures allow the recipient to verify the sender's identity with a high degree of confidence, confirming that the message was indeed sent by the claimed sender.

Integrity is maintained with the SHA-256 hashing algorithm. Any modification of the message content during transmission will result in a mismatch between the original and derived hash values, causing the verification process to fail and alerting the recipient to potential tampering.

Non-repudiation is provided by the digital signatures, which serve as proof that the sender authored the message. Once signed, the sender cannot deny having sent the message, as the signature uniquely corresponds to their private key.

Additionally, the system ensures sender access to send messages by encrypting the message with the sender's public key alongside the recipient's encryption. This design allows the sender to decrypt and view their previously sent messages without relying on the recipient's private key, enabling secure message retrieval while preserving confidentiality and integrity.

This robust cryptographic design underscores the platform's commitment to secure, reliable, and user-centric messaging functionality.

Final thoughts

Characteristics of the PKI implemented

The implemented Public Key Infrastructure (PKI) provides multiple layers of cryptographic security, supporting user authentication, secure message exchange, and HTTPS communication. Below, we explore the key characteristics and highlight how unique cryptographic processes are managed for authentication and secure communication.

Self-Signed Certificates

The use of an internal, self-signed Certificate Authority (CA) serves as the root of trust for the system. This approach simplifies the process of certificate creation and management, eliminating the need to rely on external third parties.

RSA Key Generation

The system generates separate asymmetric RSA key pairs (2048 bits) for different purposes:

- **Authentication Keys:** Generated during user registration and stored securely in the certificates/clients directory. These keys are specific to the user's identity and facilitate secure login and identity verification.
- **Messaging Keys:** Dedicated key pairs for secure communication, also generated in the registration and stored in the certificates/Message_Userkeys directory.

Certificates for Secure Messaging

Certificates specifically tailored for message exchanges are issued and signed by the CA. These certificates bind the user's messaging public key to their identity, allowing for encrypted communication and message integrity verification.

PKI for Secure HTTPS Communication

Certificates were issued to implement HTTPS on the web server, protecting the data transmitted between users and the server. These include:

- **The Same CA Certificate:** Serves as the root of trust, ensuring that certificates issued by the system are recognized as reliable.
- **Server Certificate:** Allows the server to establish secure communications with clients via HTTPS, ensuring the confidentiality and integrity of data.
- **Private Keys:** The private keys for both the server and the CA are securely stored in the certificates directory and are never shared, preserving the integrity of the PKI infrastructure.

Certificate Security

The private keys associated with the certificates are securely stored on the server, protected by encryption algorithms derived from the user's password or generated independently for PKI functionalities.

Limited Certificate Validity

Certificates have a validity of 365 days, requiring periodic renewal to reduce the risk of long-term compromise.

Trusted Cryptographic Algorithms

The use of trusted and widely adopted algorithms, such as:

- RSA: For encryption and digital signatures.
- SHA-256: For secure hashing.
- RSA-PSS: For robust digital signatures.

Integration with User Registration and Authentication

During the registration process, two distinct pairs of private and public keys are generated for each user:

- Authentication Keys:
 - The private key for authentication is securely stored in the certificates/clients directory.
 - The public key for authentication is stored securely in the **server** and used during the login process to verify the user's identity.
- Messaging Keys:
 - A separate private key is generated for secure messaging and stored in the certificates/Message_Userkeys directory.
 - The public key for messaging is stored in the database in the msg_public_key field. This key enables other users to encrypt messages securely for the intended recipient, ensuring confidentiality.

This design ensures that the cryptographic processes for authentication and secure messaging are distinct and independent, enhancing overall security and system integrity.

System Trust and Future Expansion Support

The established infrastructure supports the inclusion of additional certificates in the future, such as client certificates for advanced functionalities, ensuring the system remains flexible and scalable.

Limitations of the Current PKI Implementation

While the implemented PKI infrastructure offers robust security and demonstrates effective cryptographic principles, it has certain limitations and constraints that could affect its deployment and scalability in production environments. Below are the identified limitations:

Lack of a Verified Certificate Authority (CA)

The Certificate Authority (CA) used in this implementation is self-signed, which limits its trustworthiness outside of the controlled environment. A self-signed CA is not inherently recognized by external systems or browsers, meaning users would encounter warnings when accessing the platform. This limitation restricts the applicability of the infrastructure to internal testing or isolated environments, as it lacks the global trust offered by certificates issued by an external verified CA.

Testing Certificates

The certificates generated for HTTPS and messaging were created specifically for testing purposes and are not verified by external authorities. These certificates lack features like revocation and external validation, making them unsuitable for production systems. This constraint is acceptable for demonstration purposes but prevents deployment in environments requiring high levels of security compliance.

Lack of Certificate Revocation Mechanism

Currently, the implementation does not provide any mechanism for revoking certificates, such as a Certificate Revocation List (CRL) or an Online Certificate Status Protocol (OCSP). If a private key or certificate becomes compromised, there is no way to invalidate it. This creates a potential security risk, as compromised certificates would still be trusted by the system, potentially allowing unauthorized access or communication.

Character Limit in Encrypted Messages

The system enforces a 190-character limit on messages due to the constraints of RSA encryption and the OAEP padding scheme used. This restriction arises from the key size (2048-bit RSA) and the nature of public-key encryption, which is not optimized for encrypting large amounts of data. While this design ensures secure encryption, it limits the flexibility of the messaging functionality and restricts its usability for users who need to exchange larger pieces of information.

Fixed Key Sizes

The current implementation uses a 2048-bit RSA key for messaging and a 4096-bit RSA key for the CA. While these key sizes are considered secure at present, they may become insufficient with advances in computing, such as the development of quantum computers. This limitation highlights the need for adaptability in key sizes to future-proof the PKI infrastructure.

Centralized Key Storage

The private keys for authentication and messaging are stored on the server in directories such as `certificates/clients` and `certificates/Message_Userkeys`. Although these directories are protected, centralized storage introduces a single point of failure. If the server is compromised, all private keys could potentially be exposed. To mitigate this, more advanced storage mechanisms, such as hardware security modules (HSMs) or encrypted distributed storage, would be beneficial.

Manual Management

The current PKI setup relies heavily on manual processes, such as generating, storing, and renewing certificates. This manual intervention increases the risk of human error and reduces the scalability of the system. As the number of users or certificates grows, this approach would become increasingly impractical and error-prone, highlighting the need for automation.

No Message Encryption Compression

Messages are encrypted directly without being compressed first, which exacerbates the issue of the character limit in RSA encryption. Compression techniques could reduce the size of messages, allowing the system to handle slightly larger inputs within the constraints of the encryption scheme. Without this feature, users are limited to the small payload size enforced by the RSA encryption limits.

Limited User Certificate Management

User certificates are generated and stored on the server but are not made available for users to utilize across multiple devices. This limitation reduces flexibility, as users cannot access their certificates for purposes such as setting up secure connections on other devices or systems. Providing users with their certificates or enabling secure transfer would improve usability and expand the system's potential applications.

Summary

The current PKI implementation provides a foundational level of security and demonstrates the effective application of cryptographic principles. However, the limitations identified highlight the need for enhancements to improve its usability, scalability, and security. Addressing these constraints would make the system more robust and better suited for deployment in production environments.

Potential Improvements for the Current PKI Implementation

While the current PKI implementation provides robust security for user registration, login, and message exchange, there are potential improvements that could enhance usability, scalability, and performance.

- **Cross-Device or Multi-Browser Login:**

Currently, users may be restricted to using the accounts that they create on a single device or browser due to the local storage of private keys and certificates. To enable cross-device or multi-browser login, the system could implement device detection. When a login is attempted from a new device, the system would send a verification code to the user's registered email or phone. After entering the code, the device would be authorized, allowing secure access. This ensures security while enabling flexibility for users to log in across multiple devices.

- **Real-Time Messaging with WebSockets:**

Currently, in order to visually see the exchange of messages, you have to refresh the page. To improve this situation, it could be replaced by WebSocket-based communication. This allows for instant delivery and display of messages without requiring a page reload and reduced server load.

- **Certificate Revocation and Renewal:**

Introduce a Certificate Revocation List (CRL) or an Online Certificate Status Protocol (OCSP) to handle cases where user certificates are compromised or expired. Implement automatic renewal mechanisms for certificates close to expiration, ensuring seamless user access.

- **Support for Group Messaging:**

Extend the system to support secure group communication

- **Increase the number of characters allowed in the messages**

To improve the system's flexibility, the maximum number of characters allowed in messages could be increased. This would enable users to exchange more detailed information in a single message.

Applicability and Use Cases of the Developed PKI

The PKI implemented in this project provides a robust framework for ensuring secure communication, authentication, and data integrity. While designed primarily for demonstration and testing purposes, it has practical applications in several scenarios, each leveraging its core functionalities of encryption, authentication, and trust.

- **Secure User Authentication**

The PKI ensures a secure and cryptographically robust process for user authentication. By utilizing certificates issued by an internal Certificate Authority (CA), the system effectively verifies user identities and prevents impersonation. This approach is particularly suitable for systems requiring a high level of confidence in user authentication, such as internal corporate platforms, educational portals, or private networks.

- **Encrypted Messaging Systems**

The PKI supports secure message exchanges between users, ensuring confidentiality, integrity, and authentication. Messages are encrypted using the recipient's public key and signed with the sender's private key. This guarantees that only the intended recipient can decrypt the message and confirms the sender's identity. This feature is ideal for scenarios like secure communication platforms, team collaboration tools, or sensitive information exchanges within closed environments.

- **HTTPS Communication**

The implementation of PKI for HTTPS ensures secure data transmission between the server and clients. The certificates generated allow the server to establish secure connections, protecting data in transit from eavesdropping or tampering. This makes the PKI suitable for small-scale web services, experimental applications, or internal APIs where a verified CA is not mandatory.

- **Testing and Educational Environments**

The developed PKI, being self-signed and internally managed, is highly suitable for educational purposes or proof-of-concept projects. It allows developers and learners to explore PKI concepts, test cryptographic protocols, and understand the practicalities of certificate management without the need for external dependencies.

Examples of Use Cases

- **Secure internal Messaging Platform**

The developed PKI is well-suited for enabling secure messaging within a corporate or institutional environment. Each user is issued a unique certificate, ensuring that messages are encrypted with the recipient's public key and signed with the sender's private key. This guarantees confidentiality, authentication, and integrity of communications, which is particularly valuable in industries dealing with sensitive data.

- **Private Web Applications**

For private applications like intranet portals or internal tools, the PKI facilitates secure HTTPS communication and user authentication. Certificates are issued to both the server (for enabling HTTPS) and users (for authentication), ensuring that sensitive information exchanged on the platform is encrypted and protected from unauthorized access.

- **Prototype or Experimental Systems**

Developers working on new applications or experimental systems can leverage the PKI to test secure communication and authentication processes. The infrastructure allows developers to simulate secure production environments, validate cryptographic mechanisms, and identify potential issues before deployment.

- **Educational Tool for Cryptography**

The PKI provides a practical framework for teaching cryptography concepts in academic courses or training programs. Students can gain hands-on experience with RSA key generation, certificate creation, and secure communication, building foundational knowledge for implementing secure systems.

- **Secure File Sharing**

The PKI can be used to securely share files between users by encrypting them with the recipient's public key. This ensures that only the intended recipient, with access to the corresponding private key, can decrypt and access the file. It is particularly useful for exchanging sensitive documents such as financial reports or legal agreements.

- **Small-Scale VPN Solutions**

The PKI can act as the backbone of a small-scale VPN by authenticating users and devices through certificates. This ensures secure communication between endpoints, enabling organizations to protect their internal networks from unauthorized access.

Conclusion

The present study has sought to demonstrate the potential of cryptographic principles in securing digital systems by implementing the Public Key Infrastructure (PKI) in a specific project. The project has achieved this by addressing key challenges in authentication, message confidentiality, and data integrity, thus creating a secure platform for user interactions. The use of self-signed certificates, secure messaging mechanisms, and HTTPS communication has highlighted how PKI can provide robust and scalable security.

While the project has achieved its primary goals, there are areas for improvement, such as enabling cross-device login, enhancing encryption performance, and introducing certificate revocation mechanisms. These potential enhancements, along with other scalability and usability improvements, pave the way for refining the system for broader use cases.

In conclusion, this project serves as a practical demonstration of PKI's capabilities, offering valuable insights into its implementation and real-world applicability. The project establishes a robust foundation for future developments, ensuring secure and trustworthy communication in an increasingly interconnected digital world.